

A Resident-Streaming Roofline Model for Compute-in-Memory

Abstract—From CPUs to GPUs, the Roofline model has become a textbook tool for revealing a platform’s performance ceiling and how well a workload matches it. Compute-in-memory (CIM), however, falls outside this picture because it violates the core Roofline assumption of separate compute and memory. We present a resident-streaming Roofline model for CIM, in which streaming-activation and resident-weight throughputs replace compute throughput and memory bandwidth, respectively. This follows from a symmetry we observe: conventional processors pair heterogeneous hardware with homogeneous data, whereas CIM pairs homogeneous hardware with heterogeneous data—operands residing in the array and operands streaming through it. Using NeuroSim with literature-extracted parameters, we place ten CIM technologies, from SRAM to emerging non-volatile memories, on this Roofline and quantify the demands of mainstream large language models across computation stages. The model explains why write speed is critical in CIM: like GPU bandwidth, it caps throughput whenever reuse falls below a critical value, which spans over four orders of magnitude across these memory technologies. Likewise, the throughput at which activations stream through the CIM array plays the role of GPU compute throughput and represents the true computing performance of CIM.

Index Terms—Compute-in-memory, roofline model, resident intensity, data reuse, large language models.

I. INTRODUCTION

Outline. Use the classical Roofline [1] and Fig. 1 to motivate separate accounting for establishing resident state and evaluating streaming inputs in compute-in-memory (CIM). Position the contribution relative to Jia et al.’s IMC write/compute analysis [2]: their energy-efficiency view already exposes amortization, while this paper develops role- and window-based throughput accounting. Lead from the bound to the native capability map and reuse thresholds in Figs. 3 and 4, with inference matrix stages providing concrete demand examples.

BODY TEXT RESERVED

Introduction: motivation, nearest prior work, and contribution.

Writing round 1: prose not yet drafted.

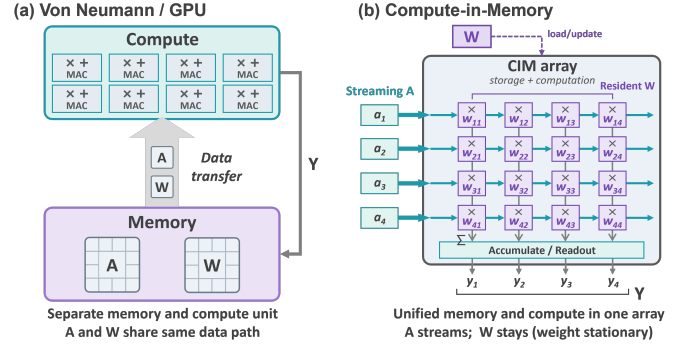


Fig. 1. Operand roles in (a) separate memory and compute units and (b) a CIM array. Streaming inputs A are evaluated against resident weights W , with a distinct path for loading or updating W .

II. A RESIDENT-STREAMING ROOFLINE

A. Operand Roles and Counting Boundaries

Outline. Define an analysis window with logical streaming-input demand Q_S and resident-write demand Q_R , in bytes, at the boundaries sketched in Fig. 1. Count initial loading only when it occurs inside the window, include every update or reload, and match ρ and τ to the same configuration, precision, and service boundaries. Explain that resident state may be weights or historical keys and values; physical encoding, conversion, and required maintenance belong in service accounting.

$$RI = \frac{Q_S}{Q_R}, \quad P = \frac{Q_S}{T} \quad [\text{Byte/s}], \quad (1)$$

where T is the window completion time; for $Q_S > 0$ and $Q_R = 0$, define $RI = \infty$.

BODY TEXT RESERVED

Counting boundaries: one compact definition paragraph and a window example.

Writing round 1: prose not yet drafted.

B. Throughput Bound and Resident Intensity

Outline. Derive the two necessary service times and use Fig. 2 to distinguish the meaning of the axes from the familiar Roofline geometry. Identify resident-bound and streaming-bound regions by the tighter term, while treating the curve as an upper bound whose attainability depends on shared resources and scheduling. Use the two compact accounting examples below to connect input throughput to useful operations and expose the effect of changing the residency window.

$$T \geq \max\left(\frac{Q_S}{\rho}, \frac{Q_R}{\tau}\right), \quad (2)$$

$$P \leq \min(\rho, \tau \text{RI}), \quad \text{RI}^* = \frac{\rho}{\tau}. \quad (3)$$

Here ρ and τ are streaming and resident service capacities in Byte/s; $\text{RI} < \text{RI}^*$ is resident-bound and $\text{RI} > \text{RI}^*$ is streaming-bound.

a) *Example to develop: useful-operation conversion:*

One logical 128×128 INT8 matrix-vector multiplication takes 128 input bytes and performs 16,384 MACs. With two useful operations per MAC, the same execution satisfies

$$\frac{32,768 \text{ OP}}{128 \text{ Byte}} = 256 \text{ OP/Byte}, \quad P_{\text{OP}} = 256P. \quad (4)$$

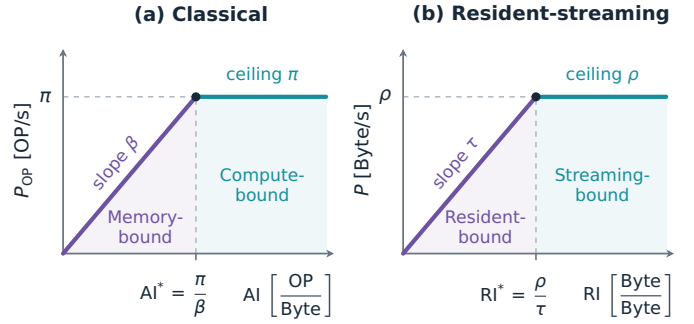
b) *Example to develop: three residency windows:* For the same matrix and U useful input vectors, $Q_S = 128U$ bytes; only the in-window loading policy changes:

Policy	Q_R [Byte]	RI
Already resident	0	∞
Load once, reuse	16,384	$U/128$
Reload before each use	$16,384U$	$1/128$.

TABLE I
RESIDENT INTENSITY AFTER ONE COMPLETE WEIGHT LOAD

Weight shape	Reuse count U					
	128	1K	16K	128K	1M	$\rightarrow \infty$
128×128	1.0	8.0	128.0	1024.0	8192.0	∞
1024×1024	0.125	1.0	16.0	128.0	1024.0	∞
4096×4096	0.0313	0.250	4.0	32.0	256.0	∞
1024×4096	0.125	1.0	16.0	128.0	1024.0	∞
4096×1024	0.0313	0.250	4.0	32.0	256.0	∞

One byte/element: $Q_S = UK$, $Q_R = NK$, $\text{RI} = U/N$. As $U \rightarrow \infty$, the single load $Q_R = NK$ stays finite. Shapes are logical matrices, not hardware tiles. 1K = 1024, 1M = 1024².



Aspect	GPU (a)	CIM (b)
Decomposition	Hardware resources	Data roles
Workload	OP, Byte	Q_S, Q_R
Service rates	π [OP/s], β [Byte/s]	ρ, τ [Byte/s]
Intensity	$\text{AI} = \text{OP/Byte}$	$\text{RI} = Q_S/Q_R$
Vertical axis	$P_{\text{OP}} = \text{OP/T}$	$P = Q_S/T$
Upper bound	$P_{\text{OP}} \leq \min(\pi, \beta \text{AI})$	$P \leq \min(\rho, \tau \text{RI})$
Left regime	Memory-bound	Resident-bound
Right regime	Compute-bound	Streaming-bound
Ridge	$\text{AI}^* = \pi/\beta$	$\text{RI}^* = \rho/\tau$

Fig. 2. Classical GPU [1] and resident-streaming CIM Rooflines with their accounting correspondence. Panels use independent linear scales; curves are upper bounds. OP and Byte denote operation and transferred-byte counts.

C. From Resident Intensity to Reuse

Outline. Define U as all input vectors served after one complete load, including the first use and allowing accumulation across requests. Use Table I and the payload relations below to connect logical matrix shape and reuse, then translate the ridge into Fig. 4 at each native service boundary. Distinguish unbounded reuse of a finite initial load from a finite, already-resident window with no writes.

$$Q_S = UKb_S, \quad Q_R = NKb_R, \quad (6)$$

$$\text{RI} = \frac{Ub_S}{Nb_R}, \quad \rho = \frac{Kb_S}{\Delta_S}, \quad (7)$$

$$\tau = \frac{NKb_R}{T_R}, \quad U^* = \frac{T_R}{\Delta_S} = \frac{Nb_R}{b_S} \text{RI}^*. \quad (8)$$

The precisions b_S, b_R are bytes per element; Δ_S is a complete-vector service interval and T_R a full-matrix load service time. At equal byte widths, $\text{RI} = U/N$ and $U^* = N\text{RI}^*$.

BODY TEXT RESERVED

Reuse interpretation and the transition to native reference services.

Writing round 1: prose not yet drafted.

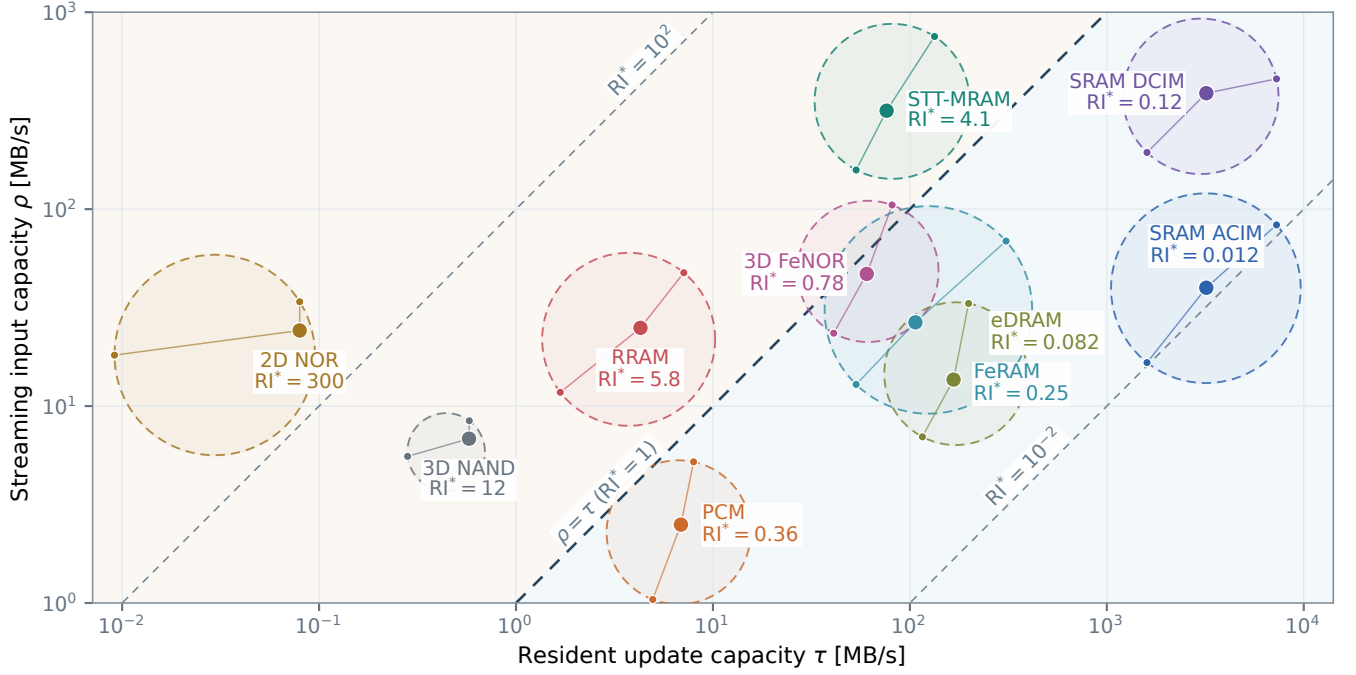


Fig. 3. Streaming capacity ρ and resident-update capacity τ for ten native reference configurations; 1 MB/s = 10^6 Byte/s. Large points are typical scenarios; small points preserve the original paired alternatives. Circles summarize only those finite scenarios, and $RI^* = 1$ is a capacity-ratio reference, not a universal workload boundary. Native sizes and resources differ.

TABLE II
RESIDENT INTENSITIES OF REPRESENTATIVE INFERENCE MATRIX STAGES

Model	Weight projections						Attention					
	QKV (U)			FFN/MoE (U)			Prefill (L)			Decode (L)		
	1K	128K	1M	16	1K	16K	1K	8K	64K	1K	8K	64K
Qwen3.5-2B	0.200	25.6	204.8	0.00347	0.222	3.6	6.0	34.0	258.0	10.0	66.0	514.0
Ministral 3 8B	0.167	21.3	170.7	0.00167	0.107	1.7	10.0	66.0	514.0	18.0	130.0	1026.0
Qwen3.6-35B-A3B	0.111	14.2	113.8	0.0130	0.833	13.3	12.0	68.0	516.0	20.0	132.0	1028.0
Hy3 (295B)	0.100	12.8	102.4	0.00477	0.306	4.9	20.0	132.0	1028.0	36.0	260.0	2052.0
Ling-1T (1000B)	0.100	12.8	102.4	0.00326	0.208	3.3	20.0	132.0	1028.0	36.0	260.0	2052.0
MiMo-V2.5-Pro (1020B)	0.0377	4.8	38.6	0.00347	0.222	3.6	35.2	214.4	1648.0	60.8	419.2	3286.4

Entries are RI; all roles use one byte/element. QKV includes any gate, excluding O. FFN/MoE counts one dense FFN or one routed expert. Prefill builds KV from empty; decode appends one token to a visible length L . 1K = 1024, 1M = 1024^2 . Ling-1T at 64K uses the fixed extension.

III. HARDWARE AND WORKLOAD CHARACTERIZATION

A. Memory-Based Reference Configurations

Outline. Introduce the ten native analog/digital reference configurations in Fig. 3, grounded in device and macro sources [3]–[12] and shared peripheral assumptions [13]–[15]. Explain the three paired engineering scenarios and complete service accounting, retaining native matrix dimensions, resources, and update organizations. Read absolute capacity and capacity ratio separately; these are literature-grounded estimates rather than equal-area chip comparisons.

BODY TEXT RESERVED

Native configurations and complete service accounting.

Writing round 1: prose not yet drafted.

B. Inference Matrix Workloads

Outline. Use the six fixed model configurations [16]–[21] as structural examples for Table II, counting common inputs once per matrix stage at one byte per element. Explain QKV intensity U/N_{proj} and FFN intensity $U(D + F)/(3DF)$ using output width N_{proj} (including any gate), hidden width D , and dense-FFN or single-expert width F . For grouped-query attention, use the native head sharing and possibly unequal query/key and value widths: prefill creates each token's KV once, whereas decode appends one token and treats historical KV as resident.

IV. REUSE REQUIREMENTS AND DESIGN IMPLICATIONS

A. Critical Reuse Across Configurations

Outline. Use Fig. 4 to show that typical full-load thresholds span about 1.6 to 38,867 input vectors, then develop one concrete crossing: at the illustrative reuse $U = 128$, STT-MRAM is resident-bound in the short and typical scenarios (about 181 and 133), but streaming-bound in the long scenario (about 95). Connect the native threshold to Qwen3.5-2B and MiMo-V2.5-Pro QKV stages using the existing sequential full-tile service reference, for which both sides accumulate the same tile count. Keep this mapping as a component-service bound, with output reduction and other system costs outside its scope.

For native 128×128 tiles, let $m_K = D/128$, $m_N = N_{\text{proj}}/128$, and $M = m_K m_N$:

$$T_{S,\text{comp}} = MU\Delta_S, \quad T_{R,\text{comp}} = MT_R, \quad (9)$$

$$\rho_{\text{eff}} = \rho/m_N, \quad \tau_{\text{eff}} = \tau, \quad (10)$$

$$\text{RI}_{\text{eff}}^* = \frac{\text{RI}^*}{m_N} = \frac{U^*}{N_{\text{proj}}}. \quad (11)$$

The factor M cancels at service balance. This reference uses one engine, full tiles without padding, and equal U per tile; input replay is included in $T_{S,\text{comp}}$.

BODY TEXT RESERVED

Explain the threshold span and one scenario crossing.

Writing round 1: prose not yet drafted.

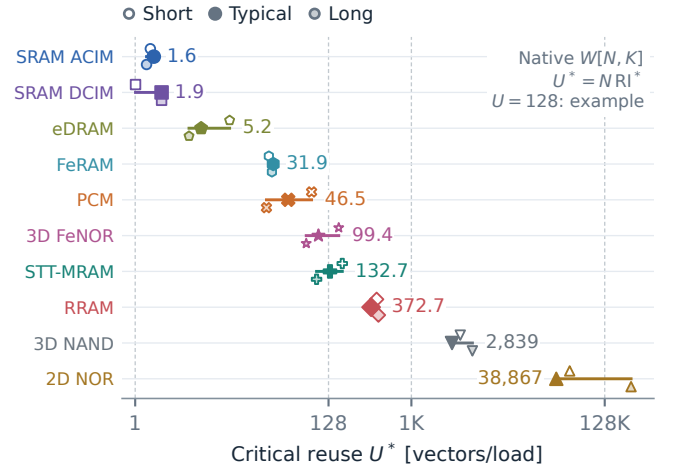


Fig. 4. Full-load critical reuse $U^* = T_R/\Delta_S = NRI^*$ at equal byte widths; each row uses its native $W[N, K]$. Numbers give typical values; segments span only three paired scenarios. Dashed lines mark illustrative reuse counts, not universal balance points. 1K = 1024.

B. Design Implications

Outline. Turn a target reuse count into an allowable full-load service time using (12), making Fig. 4 a design requirement rather than a ranking. Revisit Fig. 3 to explain why a lower threshold alone does not imply higher performance: streaming capacity still sets the upper plateau. Contrast weight residency with growing attention state in Table II; KV append requires its own update-service boundary before applying any full-load threshold.

$$T_R \leq U_{\text{target}} \Delta_S \iff U_{\text{target}} \geq U^*. \quad (12)$$

This is a necessary service-balance condition for reaching the streaming ceiling, not a guarantee of realized throughput.

BODY TEXT RESERVED

Turn a target reuse count into a resident-service requirement.

Writing round 1: prose not yet drafted.

V. CONCLUSION

Outline. Close with the role-based connection between in-window resident updates and streaming evaluation established by (3). Use Fig. 4 to emphasize the practical outcome: hardware service asymmetry becomes an explicit reuse requirement for residency and update decisions.

BODY TEXT RESERVED

Final conclusion: one short paragraph.

Writing round 1: prose not yet drafted.

REFERENCES

- [1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [2] H. Jia, M. Ozatay, Y. Tang *et al.*, "Scalable and Programmable Neural Network Inference Accelerator Based on In-Memory Computing," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, 2022.
- [3] K. Xiao, X. Cui, X. Qiao *et al.*, "A 28nm 32Kb SRAM Computing-in-Memory Macro With Hierarchical Capacity Attenuator and Input Sparsity-Optimized ADC for 4b Mac Operation," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2023.
- [4] J. Oh, C.-T. Lin, and M. Seok, "D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm², 1005-Kb/mm² Digital 6T-SRAM-Based Compute-in-Memory Macro Supporting 1-to-8b Fixed-Point Arithmetic in 28-nm CMOS," in *Proc. ESSCIRC*, 2023, pp. 413–416.
- [5] Infineon Technologies, *S29GL01GS, S29GL512S, S29GL256S, S29GL128S — 128 Mb / 256 Mb / 512 Mb / 1 Gb GL-S MIRRORBIT Flash, Parallel, 3.0 V, Military*, 2024, datasheet 002-18741 Rev. *E.
- [6] H.-T. Lue, P.-K. Hsu, M.-L. Wei *et al.*, "Optimal Design Methods to Transform 3D NAND Flash into a High-Density, High-Bandwidth and Low-Power Nonvolatile Computing in Memory (nvCIM) Accelerator for Deep-Learning Neural Networks (DNN)," in *Proc. IEEE International Electron Devices Meeting (IEDM)*, 2019, pp. 38.1.1–38.1.4.
- [7] W. Wan, R. Kubendran, C. Schaefer *et al.*, "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, pp. 504–512, 2022.
- [8] H. Li, Z. Chai, W. Dong *et al.*, "A lossless and fully parallel spintronic compute-in-memory macro for artificial intelligence chips," *Nature Electronics*, vol. 8, pp. 1046–1058, 2025.
- [9] M. Le Gallo, R. Khaddam-Aljameh, M. Stanisavljevic *et al.*, "A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference," *Nature Electronics*, vol. 6, pp. 680–693, 2023.
- [10] X. Ma, S. Deng, J. Wu *et al.*, "A 2-Transistor-2-Capacitor Ferroelectric Edge Compute-in-Memory Scheme with Disturb-Free Inference and High Endurance," *IEEE Electron Device Letters*, vol. 44, no. 7, pp. 1088–1091, 2023.
- [11] J. Song, X. Tang, H. Luo *et al.*, "A 4-bit Calibration-Free Computing-In-Memory Macro With 3T1C Current-Programmed Dynamic-Cascade Multi-Level-Cell eDRAM," *IEEE Journal of Solid-State Circuits*, 2024.
- [12] Y. Zhou, Y. Yang, L. Chen *et al.*, "3D Vertical FeFET Array with Record Endurance ($> 10^{12}$), Fast Writing ($\pm 2V$, 20 ns), Disturb Immunity, and Kb-scale Verification for High Density 1T RAM," in *Proc. Symposium on VLSI Technology and Circuits*, 2026, paper T11.1.
- [13] F. Tang, Q. Ma, Z. Shu, Y. Zheng, and A. Bermak, "A 28 nm CMOS 10 bit 100 MS/s Asynchronous SAR ADC with Low-Power Switching Procedure and Timing-Protection Scheme," *Electronics*, 2021.
- [14] B. Zimmer, S. O. Toh, H. Vo *et al.*, "SRAM Assist Techniques for Operation in a Wide Voltage Range in 28-nm CMOS," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2012.
- [15] Y. Yokoyama, K. Nii, Y. Ishii, S. Tanaka, and K. Kobayashi, "Disturbance Aware Dynamic Power Reduction in Synchronous 2RW Dual-Port 8T SRAM by Self-Adjusting Wordline Pulse Timing," *IEEE Journal of Solid-State Circuits*, 2023.
- [16] Qwen, "Qwen3.5-2B: model configuration," 2026, fixed repository snapshot, rev. 15852e8c. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.5-2B/resolve/15852e8c16360a2fea060d615a32b45270f8a8fc/config.json>
- [17] Mistral AI, "Minstral 3 8B (2512): model configuration," 2026, fixed repository snapshot, rev. d4883f9b. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/mistralai/Minstral-3-8B-Base-2512/resolve/d4883f9b36aa2e5d775730d3fdb3d30de51a8ef/config.json>
- [18] Qwen, "Qwen3.6-35B-A3B: model configuration," 2026, fixed repository snapshot, rev. 995ad96e. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen3.6-35B-A3B/resolve/995ad96eacd98c81ed38be0c5b274b04031597b0/config.json>
- [19] Tencent, "Tencent Hy3 (295B): model configuration," 2026, fixed repository snapshot, rev. a960ebc3. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/tencent/Hy3/resolve/a960ebc3da325ba167f069f76c41eb62c9280d22/config.json>
- [20] inclusionAI, "Ling-1T: model configuration," 2026, fixed repository snapshot, rev. 268f2aa7. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/inclusionAI/Ling-1T/resolve/268f2aa76aa8cf5b36ca33805481cea91133327e/config.json>
- [21] Xiaomi MiMo, "MiMo-V2.5-Pro: model configuration," 2026, fixed repository snapshot, rev. 21d1ecfe. Accessed: Sep. 23, 2026. [Online]. Available: <https://huggingface.co/XiaomiMiMo/MiMo-V2.5-Pro/resolve/21d1ecfed7bd70f31be25ca49d7edd21f003659/config.json>